MDForge: Agentic Molecular Dynamics Pipeline Design under Sparse Simulator Feedback

Zehong Wang[1] **Yijun Ma**[1] **Connor R. Schmidt**[1] **Tianyi Ma**[1] **Weixiang Sun**[1]
Ziming Li[2] **Xiaoguang Guo**[2] **Chuxu Zhang**[2] **Matthew J. Webber**[1] **Yanfang Ye**[1,†]
[1] University of Notre Dame [2] University of Connecticut
[†] Corresponding Author
<zwang43,yye7>@nd.edu

Abstract

Molecular dynamics (MD) is the canonical in-silico method for atomistic molecular science, simulating molecular behavior from first-principles physics. Designing an MD pipeline for a new system requires substantial expert knowledge: running it on even one molecule is expensive, ruling out trial-and-error. We automate this expert pipeline-design process with an LLM agent. Unlike existing MD agents that orchestrate a predefined tool set, we treat pipeline design as open-ended code generation in which the agent's behavior is reshaped online by verbal reward. Specifically, we build MDForge, an LLM agent whose in-context update rule densifies the sparse reward via a multi-agent debate among physics experts. On three SAMPL host–guest binding free-energy benchmarks (CB[7], OAH, CBClip), MDForge automatically designs MD pipelines competitive with human experts. Deployed on a library of unseen candidate guests, its CB[7] pipeline discovers a novel binder that wet-lab competition NMR confirms is a high-affinity, picomolar CB[7] binder ($K_a \approx 8 \times 10^{12}$ M^{-1}). Our data and code are available at https://github.com/Zehong-Wang/MDForge.

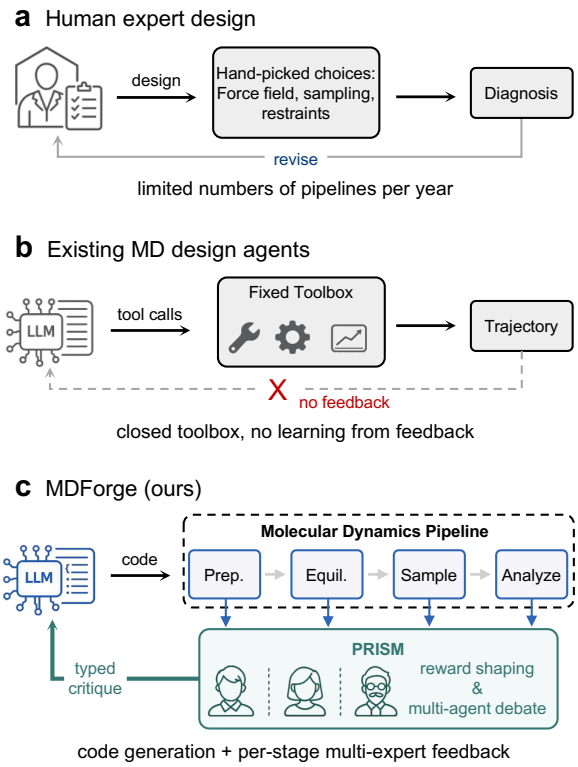


Figure 1: **Three paradigms for MD pipeline design.** (a) A human expert hand-picks each stage and iteratively revises. (b) Existing LLM agents for MD design call a fixed MD toolbox with no run-time feedback. (c) MDForge emits the pipeline as code and refines it via PRISM, a multi-expert debate over per-stage diagnostics that returns a typed critique.

1 Introduction

Molecular dynamics (MD) simulation has long been the canonical in-silico method for studying molecular behavior at atomistic resolution (Karplus and McCammon, 2002; Hollingsworth and Dror, 2018). By integrating first-principles equations of motion, MD produces atomistic trajectories from which a researcher can understand binding affinities, conformational ensembles, reaction pathways, and material properties. Several of these quantities are accessible to wet-lab measurement only at considerable expense and time; others, such as the transient conformational states populated during an enzymatic catalytic cycle, are not directly observable at all. These properties have made MD a mainstay of biology, drug discovery, and chemistry for decades (Behler, 2021; Unke et al., 2021).

Designing an MD pipeline for a new molecular system typically requires the work of trained scientists, and the throughput of new pipelines is correspondingly modest (Mey et al., 2020). A free-energy calculation illustrates this: it involves joint specification of a binding-pose hypothesis, force-field parameterization, equilibration schedule, sampling protocol, restraints, and an estimator. These

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 15482–15507
October 24-29, 2026 ©2026 Association for Computational Linguistics

choices interact non-trivially and few are universal: a pipeline tuned for one system class rarely transfers, because the dominant physics differs across system types (Mobley and Gilson, 2017; Schindler et al., 2020). The recent surge of AI-driven molecular predictors does not remove this need. A data-driven predictor outputs a target property value (*e.g.*, a binding affinity) (Merchant et al., 2023; Ross et al., 2022; Wang et al., 2026a; Ye et al., 2026) but does not produce the atomistic trajectory MD does, so it cannot supply the mechanistic account that physics-based simulation is invoked for in the first place. Its applicability is also bounded by the chemical space it was trained on: the model has no foothold on a system class for which no large labeled corpus exists (Wu et al., 2018; Yang et al., 2019), and on inputs outside its training distribution its predictions degrade silently (Bender and Cortés-Ciriano, 2021; van Tilborg et al., 2022). MD therefore remains indispensable for mechanistic understanding (Bottaro and Lindorff-Larsen, 2018), but designing its pipeline for a new system is an expert task.

In this work, we aim to design an agentic AI system that can automate the MD design by replicating the work of a trained expert. Faced with a new molecular system, the expert (Cournia et al., 2017) first inspects its chemistry, charges, rigidity, and binding mode, and these observations dictate every downstream choice: the force-field family, the equilibration schedule, the sampling protocol, the restraint scheme, and the estimator. The pipeline is then run; the expert reads the diagnostics it returns (divergence traces, free-energy convergence plots, restraint-release artifacts), identifies which subsystem misbehaved, and revises the pipeline for the next trial. Several recent LLM agents target this automation. For example, MDCrow (Campbell et al., 2026) wraps a general-purpose MD toolset (force-field setup, simulation, trajectory analysis) in LangChain-style tool calls (Yao et al., 2023); MDAgent (Ma et al., 2026b) extends the pattern with a memory module that reuses parameter choices and analytical logic from prior tasks (Zhao et al., 2024; Chen et al., 2024); DynaMate (Guilbert et al., 2025) ports the same tool-calling pattern to binding free-energy workflows. Yet none of these systems matches exactly what an MD expert does. Their tool-calling resembles the expert's selection of pipeline pieces, but only from a fixed toolbox, narrowing what the expert can otherwise compose. Likewise, none of them uses the feedback the workflow returns, yet that feedback (though sparse) is what the expert depends on to refine the pipeline.

To tackle both gaps, we propose **MDForge**, an LLM-driven agent that frames MD pipeline design as open-ended code generation (Wang et al., 2024a) under verbal reinforcement learning (Shinn et al., 2023). Code generation matches the expert's actual action space, which is not a preregistered toolbox but whatever the new system asks for. Verbal RL matches the expert's iteration habit, where each trial's diagnostics drive the next pipeline. This framing surfaces the central technical challenge of the paper: building an agent that can learn from very few feedback signals. Each signal arrives only after a full MD workflow run, whose GPU-hour cost confines each task to a small trial budget, far too limited for the agent to iteratively update behaviors in a typical approach (Wang et al., 2026b; Chen et al., 2026; Gupta et al., 2025).

At the heart of MDForge is Process-Reward Interpretation via Subsystem Mediation **(PRISM)**, an in-context update rule that turns the handful of terminal rewards into a dense, typed learning signal along two axes. First, PRISM exploits the staged nature of an MD pipeline (preparation, equilibration, production sampling, analysis): it harvests per-stage diagnostics from the simulator's intermediate outputs, so the agent receives feedback at every stage boundary rather than only at the end of the run (Lightman et al., 2024; Uesato et al., 2022; Wang et al., 2024b). Second, PRISM launches a panel of physics experts (force field, sampling, analysis) to debate each diagnostic (Du et al., 2024) and produce a typed, subsystem-attributable critique that reshapes MDForge's behavior, surfacing the kind of physical interpretation only experts can provide. Empirically, MDForge produces pipelines comparable to expert hand-designs on three SAMPL host–guest binding free-energy benchmarks (Muddana et al., 2014; Yin et al., 2017) (CB[7], OAH, and CBClip). The best AI-designed CB[7] pipeline, applied to a library of unseen candidate guests, discovers a novel binder confirmed by wet-lab competition NMR to be a high-affinity (picomolar) CB[7] binder ($K_a \approx 8 \times 10^{12}$ M^{-1}).

2 Related Work

Molecular dynamics. MDForge sits atop an established physics-based MD stack rather than competing with any of its parts: alchemical FEP/TI with BAR/MBAR estimators (Bennett, 1976; Shirts and

Chodera, 2008; Mey et al., 2020), mature simulation engines (Eastman et al., 2017; Abraham et al., 2015; Case et al., 2023), and standard biomolecular force-field families. Recent neural work replaces individual slices of this stack with learned components: ML force fields (Behler, 2021; Unke et al., 2021), structure predictors (Jumper et al., 2021), and equilibrium samplers (Noé et al., 2019). MDForge automates the *workflow* itself as executable code, so the design space is a program-synthesis over the existing toolset rather than the parameter space of a fixed pipeline template.

Autonomous science agents. LLM-driven scientific agents have integrated literature search, hypothesis proposal, and code synthesis into runnable discovery pipelines across chemistry, materials, and biology (Ye et al., 2025; Boiko et al., 2023; Bran et al., 2024; Lu et al., 2024; Merchant et al., 2023). MD-specific agents have converged on a *tool-calling* pattern that orchestrates a fixed library of engines, force fields, and analysis routines under LLM control (Campbell et al., 2026; Ma et al., 2026b; Guilbert et al., 2025; Chandrasekhar and Farimani, 2025; Shi et al., 2025). MDForge instead treats MD pipeline design as open-ended code generation in the lineage of program-synthesis agents (Wang et al., 2024a; Romera-Paredes et al., 2024), operating in a regime where the supervisory signal is both sparse (one terminal reward per trial) and expensive (GPU-hours of MD execution).

See Appendix A for extended discussion.

3 Problem Setup

Task. Given a target system class $\mathcal{T} = \{s_1, \ldots, s_M\}$ of related molecular systems with experimental references $\{y_{\text{exp}}(s_m)\}$ for some target observable y (e.g., binding free-energy), the agent emits an executable MD pipeline $\pi \in \Pi$ that, applied across $\mathcal{T}$, minimizes the mean per-system prediction error $\mathcal{L}(\pi) = \frac{1}{M}\sum_m |\hat{y}_\pi(s_m) - y_{\text{exp}}(s_m)|$.

POMDP. We cast the design loop as $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, R, \gamma)$: state $\mathcal{S}$ is the design history $(\pi_{1:t}, D_{\pi_{1:t}})$ of pipelines tried and their stage-level diagnostics; the action space $\mathcal{A} = \Pi$ is the open-ended space of executable programs that emit an MD workflow over four canonical stages (Prep, Equilibration, Production, Analysis); observations $\mathcal{O} \subseteq \mathcal{V}$ are the natural-language documents the simulator returns; transitions T are deterministic, governed by physics and the toolchain; the reward R realizes only at horizon as $r^*_\pi = -\mathcal{L}(\pi)$; and $\gamma = 1$. The reward is therefore both *sparse* (one event per pipeline) and *expensive* (a GPU-hour production run per trial). Therefore, we intend to use verbal RL to solve the problem.

Definition 1 (Verbal RL)**.** With $\mathcal{V}$ the space of natural-language strings, a POMDP is *verbal* if $\mathcal{A}, \mathcal{O} \subseteq \mathcal{V}$ and the policy is an LLM with frozen parameters θ acting on a textual context $\mathcal{C}_t \in \mathcal{V}$,

$$\pi_{t+1} \sim \text{LLM}_\theta(\cdot \mid \mathcal{C}_{t+1}), \tag{1}$$

$$\mathcal{C}_{t+1} = \text{Update}(\mathcal{C}_t, \pi_t, o_t, r_t), \tag{2}$$

where Update is an LLM call folding each trial outcome (o_t, r_t) back into the context.

4 MDForge

We present MDForge, the LLM agent that instantiates the verbal RL for automatic molecular dynamics workflow design. The framework is shown in Figure 2 with the full protocol in Appendix B.

4.1 Design Rationale

Verbal RL reduces trial-to-trial learning to the context rewrite $\mathcal{C}_t \mapsto \mathcal{C}_{t+1}$, equivalently a fast-weight update of the LLM's induced state without touching its parameters (Schmidhuber, 1992; Ba et al., 2016; Schlag et al., 2021). Under an expensive reward (each trial requires GPU-hours of MD execution before producing r^*_π), the only lever is to enrich the information each reward event carries. Two general approaches densify a sparse signal: (i) split the reward across the pipeline so each part receives its own signal (Lightman et al., 2024), and (ii) attach explanatory text to each signal value (Shinn et al., 2023). An MD pipeline supplies a natural instantiation of each: it is staged along an execution sequence, yielding per-stage diagnostics, and naturally analyzed by a multi-agent panel of physics specialists, yielding critique.

4.2 PRISM: Producing the Dense Signal

PRISM (Process-Reward Interpretation via Subsystem Mediation) is the densification machinery of MDForge: it converts the single terminal scalar r^*_π into dense signals,

$$r^*_\pi \xrightarrow{\text{PRISM}} \big(D_\pi,\, c_{\text{pre}},\, c_{\text{post}}\big), \tag{3}$$

where D_π is a K-tuple of per-stage physics diagnostics extracted from the simulator (K=4 canonical stages), and $c_{\text{pre}}, c_{\text{post}}$ are typed pre- and post-execution critiques aggregated from a panel of J=3 physics specialists.

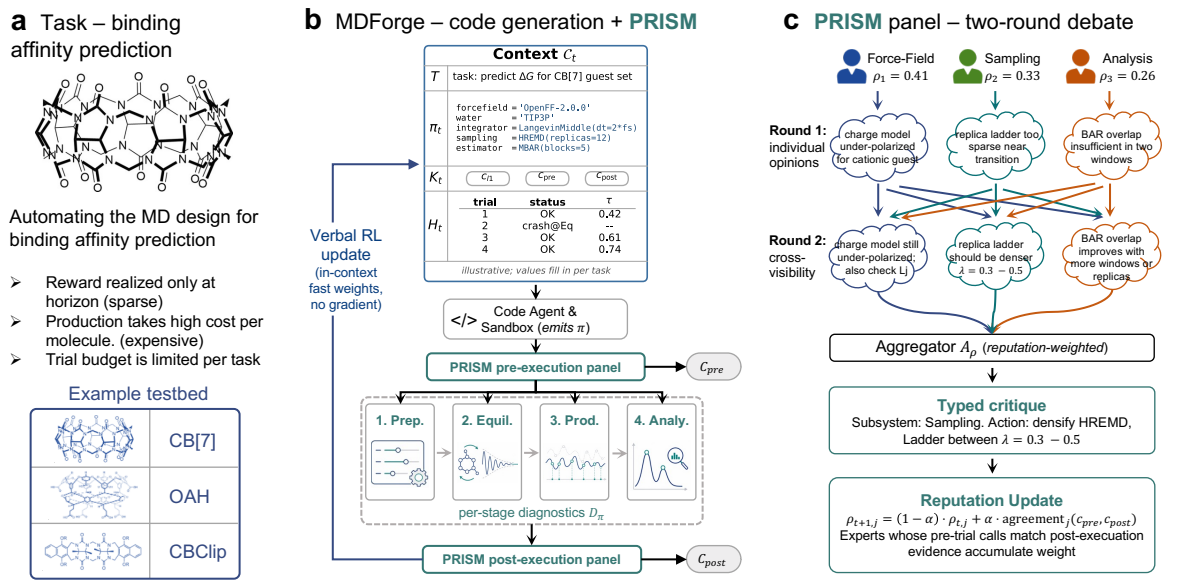


Figure 2: **Overview of MDForge.** (a) Automating MD design for binding affinity prediction, instantiated on the SAMPL CB[7], OAH, and CBClip testbeds. (b) A Code agent reads the context bundle $\mathcal{C}_t{=}\{T, \pi_t, K_t, H_t\}$ (task, current pipeline as typed code, critique set, and headline-metric trial history) and emits an executable pipeline through a sandbox. Execution proceeds through $K{=}4$ canonical stages (Preparation, Equilibration, Production, Analysis), yielding per-stage diagnostics D_π. A PRISM panel reviews π pre- and post-execution to emit typed critiques $c_{\text{pre}}, c_{\text{post}}$, which feed back into $\mathcal{C}_{t+1}$ as in-context fast-weight updates. (c) $J{=}3$ specialists (Force-Field, Sampling, Analysis) with reputations ρ_j first produce independent opinions (Round 1), then revise under cross-visibility (Round 2); a reputation-weighted aggregator $\mathcal{A}_\rho$ emits a single typed critique (subsystem + action).

Per-stage physics diagnostics. The $K{=}4$ stages (Prep, Equilibration, Production sampling, Analysis) each run against a well-defined physical objective and expose interpretable diagnostics at their boundary. We attach to each stage a physics-grounded structured diagnostic: a typed record of canonical observables (e.g., force-field self-consistency at Prep, ergodicity and PME accuracy at Production, free-energy convergence at Analysis), extracted directly from the simulator's output rather than synthesized by an LLM. Concatenated across stages, these form D_π. Only Production phase incurs extensive GPU-hour cost.

Multi-agent debate over physics subsystems. D_π is not yet actionable: a single measurement typically reflects several superimposed causes (force-field error, integrator instability, restraint misplacement, unconverged estimator) that no generic critic can disentangle. We delegate interpretation to the panel of $J{=}3$ specialist LLM agents, holding fixed, non-overlapping jurisdictions over canonical MD subsystems: *Force Field*, *Sampling*, and *Analysis*. They deliberate in two rounds with cross-visibility (Du et al., 2024), and an aggregator $\mathcal{A}_\rho$ collapses their opinions into a single typed critique, weighted by per-expert reputations $\rho = (\rho_1, \ldots, \rho_J)$ so the panel can downweight historically-miscalibrated specialists rather than equal-averaging them with reliable ones; the update rule for ρ is given in §4.3. Before the panel sees π, a tool-using Engineer agent debugs engineering faults (uncaught exceptions, missing files, mis-called APIs) in a sandbox without altering methodological choices, reserving panel deliberation for failures admitting physical attribution.

The panel is invoked at two points per trial: pre-execution it reviews π and produces c_{pre}, a cheap screen the multi-agent system can act on before burning extensive running cost; post-execution it reviews the pipeline's execution results B_π (predicted free energies with the corresponding accuracy and ranking metrics), producing

$$c_{\text{post}} \;=\; \mathcal{A}_\rho\big(\,\pi,\; B_\pi\,\big). \tag{4}$$

Together with D_π, the pair $(c_{\text{pre}}, c_{\text{post}})$ completes the PRISM densification map of Equation (3). The panel is advisory: only hard signals (Layer-1 rejection, divergence, timeout) gate execution.

4.3 Code Agent Update and Reputation Loop

Code agent update. Before the first trial, the panel holds a one-off design discussion over the task description T (with web-search access), and its

aggregated recommendations seed the Code agent's initial proposal. After each subsequent trial the Code agent regenerates the pipeline conditioned on the context bundle

$$\mathcal{C}_t \;=\; \{\, T,\; \pi_t,\; K_t,\; H_t \,\}, \tag{5}$$

where $K_t = \{c_{l1,t},\, c_{\text{pre},t},\, c_{\text{post},t}\}$ is the critique set for trial t (Layer-1 static check, pre-execution panel, post-execution panel on the benchmark) and H_t is a compact trial history over all earlier trials (per-stage statuses and headline benchmark metrics). The per-stage diagnostic D_{π_t} and terminal reward $r^*_{\pi_t}$ enter via the narrative text of $c_{\text{post},t}$ and the headline metrics in H_t; the reputation ρ_t does not enter the Code agent's view directly and only shapes the aggregated critique through $\mathcal{A}_\rho$. The prior pipeline π_t is included in $\mathcal{C}_t$ so revisions can remain localized edits when feasible rather than wholesale rewrites; the full edit protocol and prompt template are in Appendix B.

Reputation loop. A slow per-task loop maintains ρ_t by per-expert agreement between $c_{\text{pre},t}$ and $c_{\text{post},t}$: experts whose pre-trial calls are validated by post-execution evidence accumulate weight within the task. The update rule and its in-task convergence are in Appendix B.

5 Experiments

5.1 Setup

Task. We test MDForge on the SAMPL host–guest binding free-energy challenges (Muddana et al., 2014), a widely used MD benchmark and the standard tractable proxy for the protein–ligand binding problem that drives drug discovery. A rigid macrocyclic host plays the role of the protein pocket, and the task is to compute the binding free-energy $\Delta\hat{G}$ of a small molecule guest against a known experimental reference. The host–guest setting preserves the thermodynamic machinery of protein–ligand binding (water displacement, ion solvation, anharmonic guest sampling) while removing protein flexibility, so accuracy here is a necessary precondition for the full-protein setting and a fair stress test of an MD design agent. We evaluate on three hosts of distinct chemistry: *CB[7]* (SAMPL4), *OAH* (SAMPL4), and *CBClip* (SAMPL5). For each host the guests are split into a 4-guest training set (visible to the verbal RL feedback) and the remainder as a held-out test set. The docked pose for each guest is supplied; the experimental reference ΔG_{exp} is held aside from the agent. We use a cheap-MD configuration: each guest evaluation runs in ≈ 2 GPU-hours on a single A40 node.

Methods. Baselines form a capability ladder organized by the type of feedback available during pipeline construction: (1) *No feedback*: one-pass code generation with no critique and no execution signal; (2) *LLM critic* (Madaan et al., 2023): an auxiliary LLM reviews and rewrites the draft, but no code is executed; (3) *Step-level feedback* (Yao et al., 2023): tool calls return intermediate results during reasoning, enabling partial in-trial recovery but no memory across trials; (4) *Trial-level feedback* (Shinn et al., 2023): a natural-language summary of each completed trial's outcome conditions the next trial; and (5) *MDForge*: trial-level feedback augmented with PRISM (stage diagnostics and multi-expert debate). Each method runs until $N{=}5$ successful trials accumulate per host.

5.2 Main Results

Overview Results. Table 1 separates two questions: *can the agent produce a runnable MD pipeline*, and *how much ranking signal does it then recover*? The first already filters most of the ladder: No-feedback and LLM-critic baselines fail at coding on every host (0/5), Step-level feedback succeeds only intermittently (1 to 2 of 5), and reliable code emerges only with cross-trial memory (Trial-level feedback and MDForge, 5/5 everywhere). Among methods that do code, MDForge attains a held-out Kendall τ of 0.56 on CB[7] and 0.47 on CBClip against 0.24 and 0.20 for the Trial-level baseline, *more than doubling the ranking signal that transfers from the 4-guest training set to held-out guests*. On CBClip this places MDForge in the performance band of the SAMPL5 BEDAM and SOMD human submissions (Yin et al., 2017). OAH is an information-limited exception ($n_{\text{held}}{=}5$, narrow ΔG window): all coding methods cluster at $\tau \approx 0.20$. We use rank-based metrics because MD predictions carry method-specific force-field offsets (e.g., GAFF over-binds cationic CB[7] guests).

Per-Trial Results. Table 2 reports held-out Kendall τ for each of the $N{=}5$ successful trials per host, rather than the selected trial alone. The separation between MDForge and the baselines is a property of the whole distribution, not of the selection rule: on every host the MDForge trial-mean is roughly double the best baseline trial-mean (0.40

Host	Method	Runnable	Train (4 guests)			Test (held-out guests)		
			R^2	Spearman ρ	Kendall τ	R^2	Spearman ρ	Kendall τ
CB[7]	No feedback	×	–	–	–	–	–	–
	LLM critic (Madaan et al., 2023)	×	–	–	–	–	–	–
	Step-level feedback (Yao et al., 2023)	◑	0.16	0.40	0.33	0.14	0.21	0.16
	Trial-level feedback (Shinn et al., 2023)	✓	0.29	0.40	0.33	0.32	0.32	0.24
	MDForge	✓	**0.68**	**0.80**	**0.67**	**0.58**	**0.68**	**0.56**
	w/o Multi-expert debate	✓	0.73	0.80	0.67	0.53	0.33	0.33
	w/o Stage diagnostics	✓	0.34	0.40	0.33	0.45	0.10	0.16
OAH	No feedback	×	–	–	–	–	–	–
	LLM critic (Madaan et al., 2023)	×	–	–	–	–	–	–
	Step-level feedback (Yao et al., 2023)	◑	0.23	0.40	0.33	0.13	0.20	0.20
	Trial-level feedback (Shinn et al., 2023)	✓	0.47	0.60	0.33	0.21	0.20	0.20
	MDForge	✓	**0.99**	**1.00**	**1.00**	**0.43**	**0.30**	**0.20**
	w/o Multi-expert debate	✓	0.59	0.40	0.33	0.26	0.10	0.00
	w/o Stage diagnostics	✓	0.12	0.40	0.33	0.10	−0.10	0.00
CBClip	No feedback	×	–	–	–	–	–	–
	LLM critic (Madaan et al., 2023)	×	–	–	–	–	–	–
	Step-level feedback (Yao et al., 2023)	◑	0.12	0.40	0.33	0.03	0.09	0.07
	Trial-level feedback (Shinn et al., 2023)	✓	0.54	0.40	0.33	0.09	0.26	0.20
	MDForge	✓	**0.63**	**0.80**	**0.67**	**0.54**	**0.60**	**0.47**
	w/o Multi-expert debate	✓	0.71	0.80	0.67	0.23	0.37	0.33
	w/o Stage diagnostics	✓	0.54	0.40	0.33	0.03	0.31	0.20

Table 1: **Results on SAMPL host–guest binding benchmarks**: CB[7] (n_{held}=10), OAH (n_{held}=5), CBClip (n_{held}=6), with 4 training guests selected at experimental-ΔG quintile positions. We report R^2, Spearman ρ, and Kendall τ against experimental ΔG, for the best of N=5 successful trials per host (selected by training-set τ). *Runnable* summarizes how often a method produces an executable pipeline across the N=5 trials: ✓ = all 5, ◑ = 1–2, × = none. "–" marks methods with no runnable trial on any host.

Host	Method	Trial				
		1	2	3	4	5
CB[7]	Step-level	–	–	–	0.16	0.16
	Trial-level	0.20	0.16	0.22	0.18	0.24
	MDForge	0.20	0.33	0.42	0.51	0.56
OAH	Step-level	–	–	0.20	–	0.00
	Trial-level	0.00	0.20	0.20	0.20	0.00
	MDForge	0.40	0.40	0.20	0.20	0.20
CBClip	Step-level	–	–	–	0.07	0.07
	Trial-level	0.07	0.20	0.33	0.20	0.20
	MDForge	0.20	0.33	0.60	0.47	0.47

Table 2: **Per-trial held-out Kendall τ across all N=5 successful trials.** "–" marks a trial that did not yield a runnable pipeline.

Method	CB[7]	OAH	CBClip
Tool-calling, fixed hyperparameters	0.22	0.00	0.33
Tool-calling, tuned hyperparameters	0.28	0.00	0.47
MDForge (open-ended code gen.)	**0.56**	**0.20**	**0.47**

Table 3: **Open-ended code generation vs. the fixed-toolbox paradigm.** Held-out Kendall τ. The tool-calling rows reproduce the paradigm of prior MD agents (Campbell et al., 2026; Ma et al., 2026b) under matched backbone, simulator, and budget; they are not runs of those systems.

vs. 0.20 on CB[7], 0.28 vs. 0.12 on OAH, 0.41 vs. 0.20 on CBClip), and the gap between the two means is about 1.4× the MDForge standard deviation. A typical, unselected MDForge trial therefore outperforms a typical baseline trial by more than MDForge's own run-to-run noise. Selection accounts for the distance between the trial-mean and the reported endpoint (CB[7]: 0.40 → 0.56), but not for the margin over the baselines, which is already present at the mean.

Comparison Against the Fixed-Toolbox Paradigm. Prior MD agents such as MDCrow (Campbell et al., 2026) and MDAgent (Ma et al., 2026b) orchestrate a predefined toolbox rather than emitting open-ended code, and are not directly runnable on host–guest binding free-energy pipelines. We therefore reproduce their *paradigm* rather than their systems: a `tool-calling` baseline exposing 15 MD tools

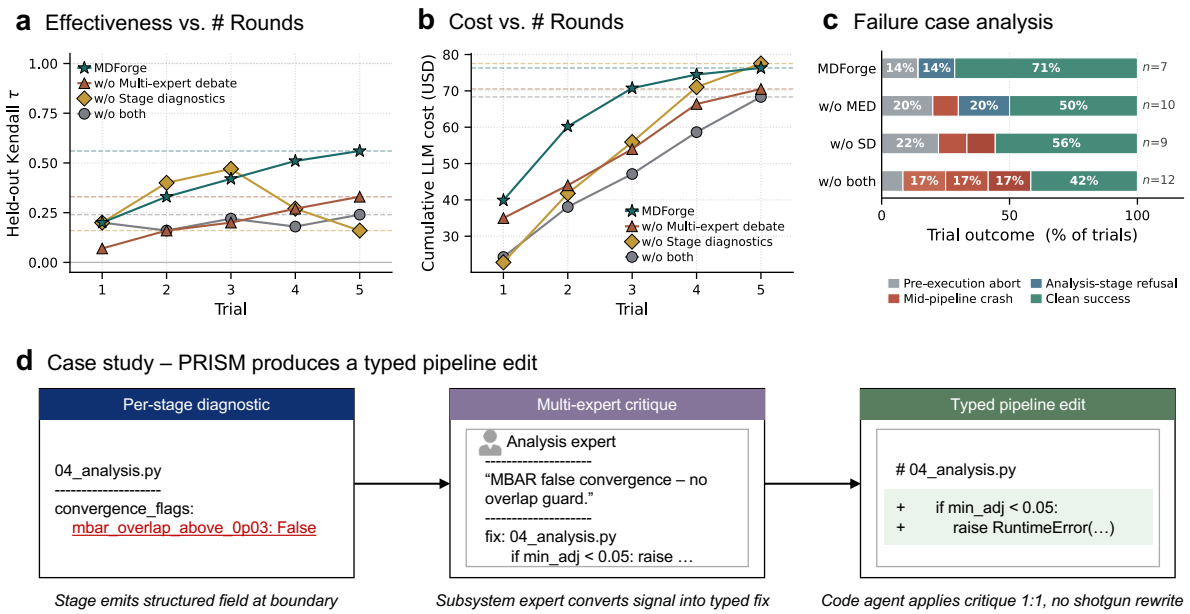


Figure 3: **PRISM mechanism on CB[7].** (a) Per-trial held-out Kendall τ over $N{=}5$ trials (CB[7] row of Table 1). MDForge improves monotonically to $\tau{=}0.56$; w/o Stage diagnostics peaks at trial 3 then collapses to 0.16, overshooting without a typed signal. (b) Cumulative *LLM token* spend ranges only \$68 to \$78 across methods, so panel (a)'s gain carries no token-cost premium. (c) PRISM types the failures: mid-pipeline crashes shrink as components are added, while analysis-stage refusal ($51\% \rightarrow 0\%$) appears only with stage diagnostics, since the convergence guard creates that category. (d) One typed signal yields one localized edit, not a shotgun rewrite: the Analysis stage emits `mbar_overlap_above_0p03 = False`, the analysis expert proposes a guard, and the Code agent applies a one-line fix at the named location.

that cover the modules a toolbox would plausibly provide (force-field and charge assignment, solvation, minimisation, NVT/NPT equilibration, APR restraints, λ-schedule construction, production, and MBAR/BAR-TI estimation), driven by the same backbone, the same simulator interface, and the same budget as MDForge. Open-ended code generation is the only variable that changes. We report two variants: one in which the agent only selects and orders tools, leaving their internal parameters at defaults, and one in which it may additionally tune the parameters of tools.

Table 3 shows that code generation dominates the toolbox on CB[7] (0.56 vs. 0.22) and on OAH, where both toolbox variants degenerate to $\tau{=}0.00$. On CBClip a well-tuned toolbox matches MDForge, but only reaches 0.47 through hyperparameter tuning layered on the fixed-parameter baseline ($0.33 \rightarrow 0.47$), whereas MDForge attains it directly. The gap is exactly the set of design choices a hand-enumerated toolbox cannot express.

5.3 Diagnosis

Figure 3 asks whether MDForge's CB[7] endpoint comes from the claimed mechanism: that verbal RL can turn PRISM's typed signals into localized pipeline edits.

Effectiveness (figure 3a). MDForge climbs monotonically to $\tau{=}0.56$, while the debate-only ablation peaks at 0.47 in trial 3 then collapses to 0.16. Without per-stage signal, the agent cannot tell a good edit from a regression and discards a working pipeline. The other two methods stall near $\tau \approx 0.20$. PRISM's value is *keeping* a high τ.

Cost (figure 3b). All four methods land within \$68 to \$78 at trial 5. MDForge's early per-trial token overhead is amortized once edits become localized rather than wholesale rewrites, so panel-(a)'s ranking gain carries no cost premium.

Failure typing (figure 3c). PRISM *types* the failures rather than lowering their count. Mid-pipeline crashes shrink from 51% w/o both to 0% on MDForge, while analysis-stage refusal appears only with stage diagnostics: it is the convergence guard explicitly declining to emit a silent MBAR false-convergence. The 56% clean-success on w/o Stage diagnostics therefore includes outcomes the guard would have refused.

Stage	*decreasing human involvement* → **Pure expert** canonical pAPRika APR pipeline	**AI + non-expert** non-expert + LLM coding assistance	**Pure AI** MDForge, autonomous
Preparation	GAFF2 + AM1-BCC; pre-charged CB[7] mol2; TIP3P + Joung-Cheatham ions; ≥12 Å pad; `taproom` dummy-atom APR geometry.	***Glide XP docking***; `antechamber` + AM1-BCC; GAFF/GAFF2/***OpenFF 2.2***; TIP3P or ***OPC***; 12 Å pad; ***no dummy geometry***.	Same GAFF2 + AM1-BCC, with ***`nz/n3→n4` widening fix***; `tleap` 18 Å pad; ***auto-scans `tleap.log`***.
Equilibrate	NVT 10→298 K (250 ps, restrained) + NPT 1 atm ramp-down (500 ps); MC barostat; HMR/SHAKE, 2–4 fs; PME.	Minimise; ***5-stage heat 50→298 K (5×10 ps)***; ***0.5 ns NPT***; optional ***"staged"*** fallback for difficult poses.	NVT heat + NPT density equilibration; HMR via OpenMM `hydrogenMass`.
Production	APR umbrella: 6 attach + 14–18 pull to $r_{\max} \approx 18$ Å; analytical release with SSC; 5–10 ns/win; HMR + 4 fs.	***5 ns unbiased NVT***, then ***z-PMF umbrella (not APR)***: ***24 windows*** $z \in [-2, +20]$ Å at 1 Å; ***xy cylinder restraint*** ($r = 2$ Å); 0.1+0.3 ns/win.	APR umbrella, ***budget-tuned***: 7 attach (λ schedule) + 14 pull at 0.5 Å; $r_{\max} = r_{\text{bound}} + 7$ Å; ***50/10 ps equil floor***; per-win velocity reseed.
Analysis	MBAR (Shirts and Chodera, 2008) on 22 windows; auto-correlation subsampling; SSC + symmetry; block-convergence flag (<0.5 kcal/mol).	`pymbar 4.0` PMF; ***Woo-Roux SSC*** from measured xy area; reports raw + corrected ΔG; ***no MBAR overlap / convergence guards***.	MBAR, ***stricter guards***: full $K \times K$ overlap; block thresh $\max(2\sigma, 0.5)$ kcal/mol; ***stat+sys uncertainty***; ***pK_a-aware protonation correction***.
Performance	ρ=**0.83**, τ=**0.68**, R^2=**0.74**	ρ=**0.61**, τ=**0.47**, R^2=**0.44**	ρ=**0.68**, τ=**0.56**, R^2=**0.58**

Table 4: **Per-stage tool selection.** Tool selection on SAMPL4 CB[7] across (i) *pure expert*, the canonical `pAPRika` APR pipeline (Slochower et al., 2019); (ii) *AI + non-expert*, a chemistry non-expert's pipeline assembled with LLM coding assistance; (iii) *pure AI*, MDForge autonomous. ***Navy bold italics*** mark choices that depart from the expert default. MDForge stays in the expert's family (GAFF2/AM1-BCC, APR, MBAR) with only reliability-flavored engineering deviations; the AI + non-expert pipeline diverges (z-PMF umbrella, no MBAR guards). Performance reports rank metrics (ρ, τ, R^2), which wash out force-field-specific systematic biases on absolute ΔG.

Case study (figure 3d). One trial follows the chain end to end: the Analysis stage emits `mbar_overlap_above_0p03 = False`; the analysis specialist proposes a guard; the Code agent applies a one-line edit at the named location, not a shotgun rewrite. Panel (c)'s blue segments aggregate this mechanism across trials, making panel (a)'s gain reproducible rather than lucky.

5.4 Does MDForge Build Like a Human Expert?

Beyond agent-vs-agent comparison, the chemistry-credibility check is whether MDForge's *pipeline* is the kind of pipeline a human MD expert would actually design. As our expert reference we use the canonical `pAPRika` APR pipeline from the Gilson lab (Henriksen et al., 2015; Slochower et al., 2019), the open-source reference implementation that has served as the *de facto* standard for host–guest free energy calculations on cucurbit[n]uril and related systems for nearly a decade. Table 4 contrasts per-stage tool selection on SAMPL4 CB[7] across three pipelines: this expert reference; an AI + non-expert pipeline that a chemistry non-expert assembled with LLM coding assistance; and MDForge running autonomously. The reading splits the two AI-touched columns in opposite directions: MDForge *stays in the expert's methodological family* (GAFF2 + AM1-BCC, APR umbrella, MBAR), departing only on reliability-flavored engineering (italicized in the table). The AI + non-expert pipeline, in contrast, *diverges to an alternative method family* (z-PMF umbrella rather than APR with dummy-atom geometry), a defensible but methodologically distinct route that LLM coding assistance plausibly leads a non-expert toward.

The *Performance* row sharpens the comparison along the same gradient. MDForge attains ρ=0.68, τ=0.56, R^2=0.58 on the SAMPL4 CB[7] guests, *recovering 78–82% of the expert's ranking utility* (ρ=0.83, τ=0.68, R^2=0.74) without a human in the loop, and *beating the AI + non-expert pipeline on all three rank-correlation metrics* (ρ=0.61, τ=0.47, R^2=0.44). Absolute-error metrics are dominated by force-field-specific systematic biases (e.g., the well-known GAFF CB[7]–cation over-binding) and therefore do not present a truly "apples-to-apples" cross-pipeline comparison.

5.5 Prospective Wet-Lab Validation

Retrospective benchmark accuracy is necessary but not sufficient: a useful pipeline must also *deploy* prospectively. We applied the best MDForge CB[7] pipeline to ten unseen candidate guests drawn from the compound bank we extracted from ChEMBL (Zdrazil et al., 2024) and DrugBank (Knox et al., 2024), ranked them by predicted $\Delta\hat{G}$, and submitted the top-1 (Bromantane, "Brom"; Figure 4a) for wet-lab measurement. Because K_a in the picomolar regime ex-

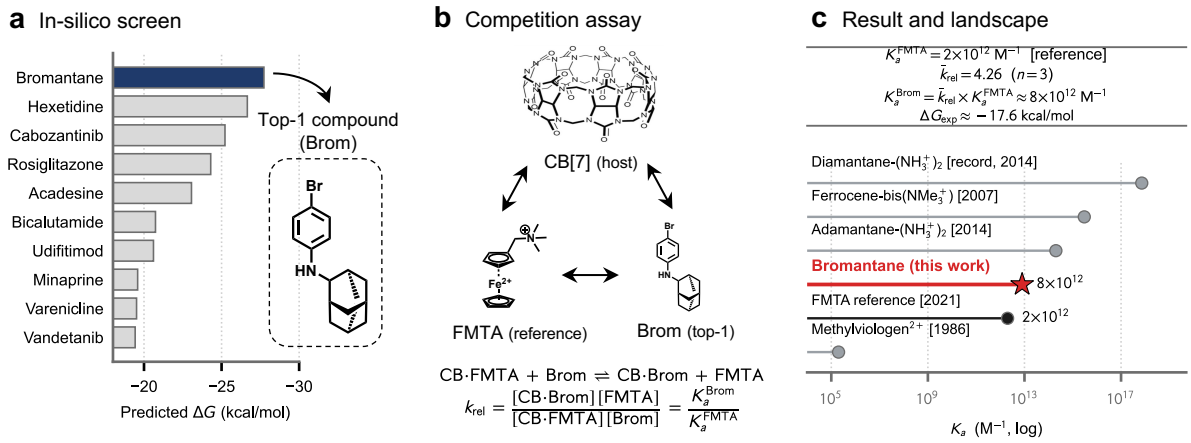


Figure 4: **End-to-end discovery from in-silico screening to wet-lab confirmation.** (a) Ten unseen candidate guests ranked by MDForge-predicted binding free-energy; top-1 is Bromantane (Brom). (b) Competition ^{1}H NMR assay: CB[7], the picomolar reference guest ferrocenylmethyl-trimethylammonium (FMTA; $K_a^{\mathrm{FMTA}} \approx 2 \times 10^{12}$ M^{-1} (Alnajjar et al., 2021)), and Brom co-equilibrate. (c) Top: measured $\bar{k}_{\mathrm{rel}} = 4.26$ (n=3) yields $K_a^{\mathrm{Brom}} \approx 8 \times 10^{12}$ M^{-1} ($\Delta G_{\mathrm{exp}} \approx -17.6$ kcal/mol). Bottom: Brom (red star) plotted against published CB[7] binders spanning K_a from 10^5 to 10^{17} M^{-1} (Cao et al., 2014; Rekharsky et al., 2007; Moghaddam et al., 2011; Liu et al., 2005; Mock and Shih, 1986).

ceeds what direct isothermal titration calorimetry can resolve, we used competition ^{1}H NMR against a reference guest of known affinity (Figure 4b). Co-equilibrating Brom with CB[7] and the canonical picomolar reference ferrocenylmethyl-trimethylammonium (FMTA; $K_a^{\mathrm{FMTA}} \approx 2 \times 10^{12}$ M^{-1} (Alnajjar et al., 2021)) establishes the exchange CB·FMTA+Brom $\rightleftharpoons$ CB·Brom+FMTA, whose relative constant $K_{\mathrm{rel}} = K_a^{\mathrm{Brom}}/K_a^{\mathrm{FMTA}}$ is read directly from the bound and unbound NMR integrations of both guests (the free-host concentration cancels). Averaging across three independent guest-ratio mixtures yields $K_{\mathrm{rel}} = 4.26$ (n=3 samples run at different molar ratios), hence $K_a^{\mathrm{Brom}} \approx 8 \times 10^{12}$ M^{-1} ($\Delta G_{\mathrm{exp}} \approx -17.6$ kcal/mol; Figure 4c, top), approximately four-fold tighter than the FMTA reference. The landscape (Figure 4c, bottom) places Brom in the picomolar high-affinity tier of published CB[7] binders, comparable to deliberately-engineered ferrocene and adamantane di-ammonium guests, while remaining ∼5 orders of magnitude below the current record holder (diamantane-bis(ammonium)) (Cao et al., 2014); all entries above Brom on this scale were obtained through years of human-driven design.

We tested only the top-1 candidate, not the other nine: this single wet-lab measurement is intended to demonstrate that MDForge can translate in-silico design into a real prospective scientific discovery, rather than to make the discovered molecule itself the primary contribution of this work. Furthermore, no claims are made regarding the expected binding affinity or validation of the full rankings across all ten hits identified from in-silico screening.

6 Conclusion

We introduced MDForge, an LLM agent that designs molecular dynamics pipelines as open-ended code under verbal RL. Its sparse terminal reward is densified by PRISM into per-stage diagnostics and a typed, subsystem-attributable expert critique. On SAMPL host–guest benchmarks, MDForge dominates other LLM-agent designs in accuracy. Its per-stage tool choices track those of the human-expert submissions. Deployed prospectively on an unseen compound library, MDForge discovered a novel CB[7] binder. Wet-lab competition NMR against the FMTA picomolar reference measures $K_a \approx 8 \times 10^{12}$ M^{-1}, placing it in the high-affinity (picomolar) regime.

Acknowledgments

This work was partially supported by the NSF under grants IIS-2533550, IIS-2321504, IIS-2217239, CNS-2426514, and CMMI-2146076, Notre Dame Strategic Framework Research Grant (2025), and Notre Dame Poverty Research Package (2025). Any expressed opinions, findings, and conclusions or recommendations are those of the authors and do not necessarily reflect the views of the sponsors.

Limitations

Benchmark scope. We benchmark MDForge on three host–guest systems from the SAMPL series (CB[7], OAH, CBClip), a restricted slice of the broader MD design space. The temporal-staging and subsystem-expert decompositions are in principle agnostic to the target system class. But the absence of equally mature open benchmarks for protein–ligand affinity, membrane protein insertion, and other binding regimes bounds our empirical reach. Future work can apply the same framework to broader binding benchmarks (e.g., FEP+, PDBbind) and to non-binding MD applications such as conformational free energy surfaces.

MD fidelity. We operate in a cheap-MD configuration (≈2 GPU-hours per guest) to keep the per-task budget tractable for systematic evaluation. The framework is in principle compatible with longer production sampling, higher-accuracy force fields, and explicit polarization. We expect the comparative method ranking in Table 1 to persist across configurations, since the limiting factor in our regime is the verbal RL update rather than the underlying MD fidelity. Verifying this at higher fidelity is left to future work.

Scope of the wet-lab demonstration. The prospective wet-lab measurement (§5.5) covers exactly one data point at the top of the predicted ranking. We do not validate the full ten-compound ranking individually, nor confirm binding affinity for any of the other predicted binders. Those measurements are out of scope here. What we do validate is that the framework retrieves a real high-affinity CB[7] binder at the top of an unseen library, end-to-end. We treat this as the right scope for a first prospective demonstration; broader prospective screens against multiple hosts are deferred to follow-up work.

Ethical Considerations

Dual-use risk. MDForge automates the design of MD pipelines for binding affinity prediction. The same capability that accelerates therapeutic discovery could in principle be repurposed to design harmful binders. Our prospective wet-lab result sharpens this concern from a theoretical possibility to an operational one. The framework's direct output is a simulation protocol (Python code), not a molecule. The locus of dual-use control therefore sits upstream (in the candidate library and the choice of target) and downstream (in the interpretation and deployment), not at MDForge itself. Responsible deployment requires governance over those layers, consistent with established norms in computational chemistry and structure-based drug discovery (Boiko et al., 2023; Bran et al., 2024).

Scope of the wet-lab demonstration. The compound bank from which the top-1 candidate was drawn was constructed in separate work and is not a curated set of pharmacologically active species. CB[7] is a synthetic macrocyclic host, not a biological target. The discovered binder therefore has no direct therapeutic use, and the experiment should not be read as a drug-discovery claim. The bank composition is governed by the originating project; the curated compound bank will be released in a follow-on publication.

Computational footprint. End-to-end MD-based virtual screening is energy-intensive. We deliberately operate in a cheap-MD configuration (≈2 GPU-hours per guest) and a small trial budget, both of which lower the per-discovery energy cost relative to traditional expert-design iteration. Useful community directions for further reducing this footprint include energy-aware scheduling, surrogate filters that defer expensive sampling, and shared baselines that quantify energy-per-prediction relative to human-designed pipelines.

Reproducibility and auditability. AI-designed MD pipelines must be auditable by domain experts before being treated as scientific instruments. MDForge produces Python code as its action rather than opaque numerical parameters, so individual pipelines are inspectable. The generated code, however, is often long and stylistically idiosyncratic, which raises the audit burden. We release the agent code, per-trial pipeline artifacts, multi-expert prompts, and the full LLM configuration. Promising future directions include pipeline-summarization tools that compress agent-generated code into expert-readable protocol descriptions, and automated provenance tracking that links each design choice back to the typed critique that motivated it.

References

Mark James Abraham, Teemu Murtola, Roland Schulz, Szilárd Páll, Jeremy C. Smith, Berk Hess, and

Erik Lindahl. 2015. GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers. *SoftwareX*.

Mohammad A. Alnajjar, Werner M. Nau, and Andreas Hennig. 2021. A reference scale of cucurbit[7]uril binding affinities. *Organic & Biomolecular Chemistry*.

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight experience replay. In *Advances in Neural Information Processing Systems*.

Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. 2019. RUDDER: Return decomposition for delayed rewards. In *Advances in Neural Information Processing Systems*.

Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. 2016. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*.

Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. 2025. ResearchAgent: Iterative research idea generation over scientific literature with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, and 1 others. 2022. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

Jörg Behler. 2021. Four generations of high-dimensional neural network potentials. *Chemical Reviews*.

Andreas Bender and Isidro Cortés-Ciriano. 2021. Artificial intelligence in drug discovery: What is realistic, what are illusions? Part 1: Ways to make an impact, and why we are not there yet. *Drug Discovery Today*.

Charles H. Bennett. 1976. Efficient estimation of free energy differences from Monte Carlo data. *Journal of Computational Physics*.

Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. 2023. Autonomous chemical research with large language models. *Nature*.

Stefan Boresch, Franz Tettinger, Martin Leitgeb, and Martin Karplus. 2003. Absolute binding free energies: A quantitative approach for their calculation. *The Journal of Physical Chemistry B*.

Sandro Bottaro and Kresten Lindorff-Larsen. 2018. Biophysical experiments and biomolecular simulations: A perfect match? *Science*.

Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2019. Exploration by random network distillation. In *International Conference on Learning Representations*.

Quintina Campbell, Sam Cox, Jorge Medina, Brittany Watterson, and Andrew D. White. 2026. MDCrow: Automating molecular dynamics workflows with large language models. *Machine Learning: Science and Technology*.

Liping Cao, Marina Šekutor, Peter Y. Zavalij, Kata Mlinarić-Majerski, Robert Glaser, and Lyle Isaacs. 2014. Cucurbit[7]uril·guest pair with an attomolar dissociation constant. *Angewandte Chemie International Edition*.

David A. Case, Hasan Metin Aktulga, Kellon Belfon, David S. Cerutti, G. Andrés Cisneros, Vinicius Wilian D. Cruzeiro, Negin Forouzesh, Timothy J. Giese, Andreas W. Goetz, Holger Gohlke, and 1 others. 2023. AmberTools. *Journal of Chemical Information and Modeling*.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2025. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*.

Achuth Chandrasekhar and Amir Barati Farimani. 2025. Automating MD simulations for proteins using large language models: NAMD-agent. *arXiv preprint arXiv:2507.07887*.

Guoxin Chen, Jie Chen, Lei Chen, Jiale Zhao, Fanzhe Meng, Wayne Xin Zhao, Ruihua Song, Cheng Chen, Ji-Rong Wen, and Kai Jia. 2026. Toward autonomous long-horizon engineering for ML research. *arXiv preprint arXiv:2604.13018*.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*.

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zoe Cournia, Bryce Allen, and Woody Sherman. 2017. Relative binding free energy calculations in drug discovery: recent advances and practical considerations. *Journal of chemical information and modeling*.

Lijie Ding, Jan-Michael Carrillo, and Changwoo Do. 2026. ToPolyAgent: AI agents for coarse-grained bead-spring topological polymer simulations. *Digital Discovery*.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*.

Peter Eastman, Jason Swails, John D. Chodera, Robert T. McGibbon, Yutong Zhao, Kyle A. Beauchamp, Lee-Ping Wang, Andrew C. Simmonett, Matthew P. Harrigan, Chaya D. Stern, Rafal P. Wiewiora, Bernard R. Brooks, and Vijay S. Pande. 2017. OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*.

Alireza Ghafarollahi and Markus J. Buehler. 2025. Automating alloy design and discovery with physics-aware multimodal multiagent AI. *Proceedings of the National Academy of Sciences*.

Michael K. Gilson, James A. Given, Bruce L. Bush, and J. Andrew McCammon. 1997. The statistical-thermodynamic basis for computation of binding affinities: A critical review. *Biophysical Journal*.

Salomé Guilbert, Cassandra Masschelein, Jeremy Goumaz, Bohdan Naida, and Philippe Schwaller. 2025. DynaMate: An autonomous agent for protein-ligand molecular dynamics simulations. *arXiv preprint arXiv:2512.10034*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*.

Rushil Gupta, Jason Hartford, and Bang Liu. 2025. LLMs for Bayesian optimization in scientific domains: Are we there yet? In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Lester Hedges, Antonia S. J. S. Mey, Charles A. Laughton, Francesco L. Gervasio, Adrian J. Mulholland, Christopher J. Woods, and Julien Michel. 2019. BioSimSpace: An interoperable Python framework for biomolecular simulation. *Journal of Open Source Software*.

Niel M. Henriksen, Andrew T. Fenley, and Michael K. Gilson. 2015. Computational calorimetry: High-precision calculation of host-guest binding thermodynamics. *Journal of Chemical Theory and Computation*.

Scott A. Hollingsworth and Ron O. Dror. 2018. Molecular dynamics simulation for all. *Neuron*.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, and 15 others. 2021. Highly accurate protein structure prediction with AlphaFold. *Nature*.

Martin Karplus and J. Andrew McCammon. 2002. Molecular dynamics simulations of biomolecules. *Nature Structural Biology*.

Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. 2026. Process reward models that think. *Transactions on Machine Learning Research*.

Craig Knox, Mike Wilson, Christen M. Klinger, Mark Franklin, Eponine Oler, Alex Wilson, Allison Pon, Jordan Cox, Na Eun (Lucy) Chin, Seth A. Strawbridge, Marysol Garcia-Patino, Ray Kruger, Aadhavya Sivakumaran, Selena Sanford, Rahil Doshi, Nitya Khetarpal, Omolola Fatokun, Daphnee Doucet, Ashley Zubkowski, and 23 others. 2024. DrugBank 6.0: the DrugBank knowledgebase for 2024. *Nucleic Acids Research*.

Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, Maxime Gazeau, Himanshu Sahni, Satinder Singh, and Volodymyr Mnih. 2023. In-context reinforcement learning with algorithm distillation. In *International Conference on Learning Representations*.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Ren Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. 2024. RLAIF vs. RLHF: Scaling

reinforcement learning from human feedback with AI feedback. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Simin Liu, Christian Ruspic, Pritam Mukhopadhyay, Sriparna Chakrabarti, Peter Y. Zavalij, and Lyle Isaacs. 2005. The cucurbit[n]uril family: Prime components for self-sorting systems. *Journal of the American Chemical Society*.

Yitao Liu, Chenglei Si, Karthik Narasimhan, and Shunyu Yao. 2025. Contextual experience replay for self-improvement of language agents. In *Annual Meeting of the Association for Computational Linguistics*.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Meiqi Guo, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, and Abhinav Rastogi. 2024. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*.

Renjie Luo, Zichen Liu, Xiangyan Liu, Chao Du, Min Lin, Wenhu Chen, Wei Lu, and Tianyu Pang. 2025. Language models can learn from verbal feedback without scalar rewards. *arXiv preprint arXiv:2509.22638*.

Tianyi Ma, Yiyue Qian, Zheyuan Zhang, Zehong Wang, Xiaoye Qian, Feifan Bai, Yifan Ding, Xuwei Luo, Shinan Zhang, Keerthiram Murugesan, Chuxu Zhang, and Yanfang Ye. 2025. AutoData: A multi-agent system for open web data collection. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations*.

Yijun Ma, Zehong Wang, Weixiang Sun, Zheyuan Zhang, Kaiwen Shi, Nitesh Chawla, and Yanfang Ye. 2026a. Policy4OOD: A knowledge-guided world model for policy intervention simulation against the opioid overdose crisis. *arXiv preprint arXiv:2602.12373*.

Zhenyu Ma, Chunyi Yang, Yuyang Song, Jingyi Zhu, Letian Yang, Limei Xu, Min Xiao, and Xukai Jiang. 2026b. MDAgent: A multi-agent framework for end-to-end molecular dynamics research. *arXiv preprint arXiv:2604.18622*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Gowoon Cheon, and Ekin Dogus Cubuk. 2023. Scaling deep learning for materials discovery. *Nature*.

Antonia S. J. S. Mey, Bryce K. Allen, Hannah E. Bruce Macdonald, John D. Chodera, David F. Hahn, Maximilian Kuhn, Julien Michel, David L. Mobley, Levi N. Naden, Samarjeet Prasad, Andrea Rizzi, Jenke Scheen, Michael R. Shirts, Gary Tresadern, and Huafeng Xu. 2020. Best practices for alchemical free energy calculations [article v1.0]. *Living Journal of Computational Molecular Science*.

David L. Mobley and Michael K. Gilson. 2017. Predicting binding free energies: Frontiers and benchmarks. *Annual Review of Biophysics*.

William L. Mock and Neng Yang Shih. 1986. Structure and selectivity in host-guest complexes of cucurbituril. *Journal of Organic Chemistry*.

Sarvin Moghaddam, Cheng Yang, Mikhail Rekharsky, Young Ho Ko, Kimoon Kim, Yoshihisa Inoue, and Michael K. Gilson. 2011. New ultrahigh affinity host-guest complexes of cucurbit[7]uril with bicyclo[2.2.2]octane and adamantane guests: Thermodynamic analysis and evaluation of m2 affinity calculations. *Journal of the American Chemical Society*.

Hari S. Muddana, Andrew T. Fenley, David L. Mobley, and Michael K. Gilson. 2014. The SAMPL4 host-guest blind prediction challenge: An overview. *Journal of Computer-Aided Molecular Design*.

Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*.

Frank Noé, Simon Olsson, Jonas Köhler, and Hao Wu. 2019. Boltzmann generators: Sampling equilibrium states of many-body systems with deep learning. *Science*.

Odhran O'Donoghue, Aleksandar Shtedritski, John Ginger, Ralph Abboud, Ali Ghareeb, and Samuel Rodriques. 2023. BioPlanner: Automatic evaluation of LLMs on protocol planning in biology. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology*.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. 2017. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative agents for software development. In *Annual Meeting of the Association for Computational Linguistics*.

Yuanhao Qu, Kaixuan Huang, Ming Yin, Kanghong Zhan, Dyllan Liu, Di Yin, Henry C. Cousins, William A. Johnson, Xiaotong Wang, Mihir Shah, Russ B. Altman, Denny Zhou, Mengdi Wang, and Le Cong. 2025. CRISPR-GPT for agentic automation of gene-editing experiments. *Nature Biomedical Engineering*.

Mikhail V. Rekharsky, Tadashi Mori, Cheng Yang, Young Ho Ko, Narayanan Selvapalam, Hyunuk Kim, David Sobransingh, Angel E. Kaifer, Simin Liu, Lyle Isaacs, Wei Chen, Sarvin Moghaddam, Michael K. Gilson, Kimoon Kim, and Yoshihisa Inoue. 2007. A synthetic host-guest system achieves avidin-biotin affinity by overcoming enthalpy-entropy compensation. *Proceedings of the National Academy of Sciences*.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature*.

Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. 2022. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*.

Christina E. M. Schindler, Hannah Baumann, Andreas Blum, Dietrich Bose, Hans-Peter Buchstaller, Lars Burgdorf, Daniel Cappel, Eugene Chekler, Paul Czodrowski, Dieter Dorsch, Merveille K. I. Eguida, Bruce Follows, Thomas Fuchss, Ulrich Grädler, Jakub Gunera, Theresa Johnson, Catherine Jorand Lebrun, Srinivasa Karra, Markus Klein, and 18 others. 2020. Large-scale assessment of binding free energy calculations in active drug discovery projects. *Journal of Chemical Information and Modeling*.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. 2021. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Jürgen Schmidhuber. 1992. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. 2016. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*.

Zhuofan Shi, Hubao A, Yufei Shao, Dongliang Huang, Hongxu An, Chunxiao Xin, Haiyang Shen, Zhenyu Wang, Yunshan Na, Gang Huang, and Xiang Jing. 2026. MDAgent2: Large language model for code generation and knowledge Q&A in molecular dynamics. *arXiv preprint arXiv:2601.02075*.

Zhuofan Shi, Chunxiao Xin, Tong Huo, Yuntao Jiang, Bowen Wu, Xingyue Chen, Wei Qin, Xinjian Ma, Gang Huang, Zhenyu Wang, and Xiang Jing. 2025. A fine-tuned large language model based molecular dynamics agent for code generation to obtain material thermodynamic parameters. *Scientific Reports*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*.

Michael R. Shirts and John D. Chodera. 2008. Statistically optimal analysis of samples from multiple equilibrium states. *The Journal of Chemical Physics*.

David R. Slochower, Niel M. Henriksen, Lee-Ping Wang, John D. Chodera, David L. Mobley, and Michael K. Gilson. 2019. Binding thermodynamics of host-guest systems with SMIRNOFF99Frosst 1.0.5 from the Open Force Field Initiative. *Journal of Chemical Theory and Computation*.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*.

Jaehyeon Son, Soochan Lee, and Gunhee Kim. 2025. Distilling reinforcement learning algorithms for in-context model-based planning. In *International Conference on Learning Representations*.

Nathan J. Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E. Kumar, Tanjin He, David Milsted, Matthew J. McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, Haegyeom Kim, Anubhav Jain, Christopher J. Bartel, Kristin Persson, Yan Zeng, and Gerbrand Ceder. 2023. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*.

Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. 2025. AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML. In *International Conference on Machine Learning*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Oliver T. Unke, Stefan Chmiela, Huziel E. Sauceda, Michael Gastegger, Igor Poltavsky, Kristof T. Schütt, Alexandre Tkatchenko, and Klaus-Robert Müller. 2021. Machine learning force fields. *Chemical Reviews*.

Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. 2022. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*.

Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. 2017. Feudal networks for hierarchical reinforcement learning. In *International Conference on Machine Learning*.

Aikaterini Vriza, Uma Kornu, Aditya Koneru, Henry Chan, and Subramanian K. R. S. Sankaranarayanan. 2026. Multi-agentic AI framework for end-to-end atomistic simulations. *Digital Discovery*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Lingle Wang, Yujie Wu, Yuqing Deng, Byungchan Kim, Levi Pierce, Goran Krilov, Dmitry Lupyan, Shaughnessy Robinson, Markus K. Dahlgren, Jeremy Greenwood, Donna L. Romero, Craig Masse, Jennifer L. Knight, Thomas Steinbrecher, Thijs Beuming, Wolfgang Damm, Edward Harder, Woody Sherman, Mark Brewer, and 10 others. 2015. Accurate and reliable prediction of relative ligand binding potency in prospective drug discovery by way of a modern free-energy calculation protocol and force field. *Journal of the American Chemical Society*.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024b. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics*.

Zehong Wang, Xiaolong Han, Qi Yang, Xiangru Tang, Fang Wu, Xiaoguang Guo, Weixiang Sun, Tianyi Ma, Pietro Lio, Le Cong, Sheng Wang, Chuxu Zhang, and Yanfang Ye. 2026a. Molecular representations in implicit functional space via hyper-networks. *arXiv preprint arXiv:2601.22327*.

Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. 2026b. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents. *arXiv preprint arXiv:2601.22311*.

Fang Wu, Weihao Xuan, Heli Qi, Hanqun Cao, Heng-Jui Chang, Zeqi Zhou, Li Erran Li, Haokai Zhao, Jian Ma, Zijian Carl Ma, Yu-Chi Cheng, Kuan Pang, Xiangru Tang, Zehong Wang, Guanlue Li, Hanchen Wang, Kejun Ying, Pan Lu, Chiho Im, and 10 others. 2026a. Proteo-r1: Reasoning foundation models for de novo protein design. In *International Conference on Machine Learning (ICML)*.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *Conference on Language Modeling (COLM)*.

Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2026b. From interactions to principles: Experience-driven self-distillation for evolving LLM agents. In *International Conference on Machine Learning*.

Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. 2018. MoleculeNet: A benchmark for molecular machine learning. *Chemical Science*.

Zhihui Xie, Jie Chen, Liyu Chen, Weichao Mao, Jingjing Xu, and Lingpeng Kong. 2025. Teaching language models to critique via reinforcement learning. In *International Conference on Machine Learning*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024a. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Fengxu Yang and Jack D. Evans. 2026. QUASAR: A universal autonomous system for atomistic simulation and a benchmark of its capabilities. *Journal of Chemical Information and Modeling*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*.

Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, Tommi Jaakkola, Klavs Jensen, and Regina Barzilay. 2019. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Xinwu Ye, Yicheng Mao, Yuxuan Liao, Jia Zhang, Yimeng Liu, Li Hao, Fang Wu, Zhiwei Li, Zehong Wang, Zhiyuan Liu, Zhenfei Yin, Li Yuan, Philip Torr, Huan Sun, Xiangxiang Zeng, Mengdi Wang, Le Cong, Shenghua Gao, and Xiangru Tang. 2026. LatentChem: From textual CoT to latent thinking in chemical reasoning. In *International Conference on Machine Learning (ICML)*.

Yanfang Ye, Zheyuan Zhang, Tianyi Ma, Zehong Wang, Yiyang Li, Shifu Hou, Weixiang Sun, Kaiwen Shi, Yijun Ma, Wei Song, and 1 others. 2025. Llms4all: A review of large language models across academic disciplines. *arXiv preprint arXiv:2509.19580*.

Jian Yin, Niel M. Henriksen, David R. Slochower, Michael R. Shirts, Michael W. Chiu, David L. Mobley, and Michael K. Gilson. 2017. Overview of the SAMPL5 host-guest challenge: Are we doing better? *Journal of Computer-Aided Molecular Design*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Barbara Zdrazil, Eloy Felix, Fiona Hunter, Emma J. Manners, James Blackshaw, Sybilla Corbett, Marleen de Veij, Harris Ioannidis, David Mendez Lopez, Juan F. Mosquera, María Paula Magariños, Nicolas Bosc, Ricardo Arcila, Tevfik Kizilören, Anna Gaulton, A. Patrícia Bento, Melissa F. Adasme, Peter Monecke, Gregory A. Landrum, and Andrew R. Leach. 2024. The chembl database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*.

Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. 2024. ReST-MCTS*: LLM self-training via process reward guided tree search. In *Advances in Neural Information Processing Systems*.

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. The lessons of developing process reward models in mathematical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*.

Alexander Zhao, Achuth Chandrasekhar, and Amir Barati Farimani. 2026. PolyJarvis: An LLM-orchestrated agent for automated all-atom molecular dynamics of amorphous homopolymers. *arXiv preprint arXiv:2604.02537*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Annual Meeting of the Association for Computational Linguistics*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*.

A Comprehensive Related Work

A.1 Molecular Dynamics

MDForge sits atop the established methodological stack of physics-based MD rather than competing with any of its parts. The binding free-energy task we target is computed by the alchemical FEP/TI family with BAR/MBAR estimators (Bennett, 1976; Shirts and Chodera, 2008; Mey et al., 2020); absolute free-energy workflows additionally rely on standard-state restraint corrections (Boresch et al., 2003; Gilson et al., 1997) and have been benchmarked extensively in the relative regime (Wang et al., 2015; Cournia et al., 2017). The executable pipelines our agent emits target mature engines (OpenMM, GROMACS, AMBER) (Eastman et al., 2017; Abraham et al., 2015; Case et al., 2023) and biomolecular force-field families (AMBER, GAFF/OpenFF, TIP3P/OPC water), which form the action vocabulary the LLM composes into rather than targets it tries to improve. A parallel line of work proposes neural surrogates for parts of the pipeline: ML force fields trained to replace classical potentials (Behler, 2021; Unke et al., 2021), end-to-end structure predictors (Jumper et al., 2021; Wu et al., 2026a), and neural samplers that draw equilibrium configurations directly (Noé et al., 2019). Each replaces a single slice (a force, a static structure, an equilibrium sample), but none yields the staged, diagnostic-emitting trajectory whose design MDForge automates. Earlier non-LLM frameworks (BioSimSpace, OpenFE, perses) (Hedges et al., 2019) also templatize parts of this workflow; MDForge instead emits the workflow itself as code, so the design space is the program-synthesis space rather than the parameter space of a fixed template.

A.2 Autonomous Science Agents

LLM-driven scientific agents integrate literature search, hypothesis proposal, code synthesis, and outer-loop evaluation into runnable discovery pipelines (Boiko et al., 2023; Romera-Paredes et al., 2024; Lu et al., 2024; Bran et al., 2024; Baek et al., 2025; Schmidgall et al., 2025; Chen et al., 2026; Ma et al., 2026a), with parallel instantiations in materials discovery and autonomous laboratories (Merchant et al., 2023; Szymanski et al., 2023), biological experiment and protocol planning (Qu et al., 2025; O'Donoghue et al., 2023), and software- or ML-engineering pipeline automation (Yang et al., 2024b; Trirat et al., 2025; Chan et al., 2025). Closest to our setting, MD-specific agents have converged on a *tool-calling* pattern in which an LLM orchestrates a fixed library of simulation engines, force fields, and analysis libraries into an executable workflow. MDCrow (Campbell et al., 2026) provides a general-purpose toolset over force-field setup, simulation, and trajectory analysis, exposed to an LLM through LangChain-style tool calls. MDAgent (Ma et al., 2026b) extends the pattern with a case-based skill-and-memory module that retrieves prior task knowledge across trials, the most ambitious recent attempt at inter-trial improvement within the tool-calling paradigm. Similar tool-calling patterns extend to binding free-energy workflows (Guilbert et al., 2025), polymer and topological MD (Zhao et al., 2026; Ding et al., 2026), NAMD-based protein simulations (Chandrasekhar and Farimani, 2025), alloy design (Ghafarollahi and Buehler, 2025), broader atomistic settings (Vriza et al., 2026; Yang and Evans, 2026), and to fine-tuned LLM variants that emit MD scripts directly (Shi et al., 2025, 2026). In contrast, MDForge treats MD pipeline design as *open-ended code generation*, placing it in the lineage of program-synthesis agents that emit arbitrary executable code rather than select from a fixed tool library (Wang et al., 2024a; Romera-Paredes et al., 2024); unlike these, MDForge operates in a regime whose only supervisory signal is sparse (one terminal reward per full pipeline run) and expensive (GPU-hours of MD execution per trial).

A.3 Verbal Reinforcement Learning

In-context learning enables an LLM agent to reshape its behavior online by manipulating context rather than parameters, requiring no gradient update. Reflexion (Shinn et al., 2023) formalized this as *verbal reinforcement learning*: an LLM agent attempts a trial, receives a textual outcome label, generates a natural-language reflection, and consumes that reflection as additional context on the next trial; recent work further formalizes this verbal-feedback channel without scalar rewards (Luo et al., 2025). Subsequent work extends this paradigm with reasoning-action interleaving (Yao et al., 2023; Madaan et al., 2023), persistent skill libraries (Wang et al., 2024a; Wu et al., 2026b), experiential memory (Zhao et al., 2024; Park et al., 2023; Liu et al., 2025), multi-agent coordination (Wu et al., 2024; Du et al., 2024; Qian et al., 2024), in-context RL (Laskin et al., 2023; Chen et al., 2021; Son et al., 2025),

textual optimization that treats verbal feedback as a gradient-like signal over prompts or programs (Yang et al., 2024a; Yuksekgonul et al., 2025), and LLM-generated dense reward or critique (Ma et al., 2024; Chen et al., 2024; Xie et al., 2025). However, all these methods rely on a supervisory signal that is both *rich* and *cheap*, typically validated on benchmarks where per-trial cost is seconds to minutes (HotpotQA, AlfWorld, code unit tests). We extend verbal RL to the opposite regime: each trial costs GPU-hours of MD execution and yields only a single scalar at horizon end.

A.4 Why Verbal RL for MDForge

Because θ is fixed, all learning is carried by the context rewrite $\mathcal{C}_t \mapsto \mathcal{C}_{t+1}$; the design of $\mathcal{C}_t$ is therefore the operative learning rule. No other update rule fits the regime: classical sparse-reward methods (reward shaping (Ng et al., 1999), Bayesian optimization (Shahriari et al., 2016), hindsight relabeling (Andrychowicz et al., 2017)) presume a feature space that Π does not admit; gradient-based RL (Schulman et al., 2017) demands orders of magnitude more rollouts; static few-shot prompting (Brown et al., 2020) absorbs no trial signal at all. Verbal RL alone (i) treats the trial signal as text, so heterogeneous per-stage diagnostics fold back in without a fixed feature representation, (ii) keeps the policy a frozen LLM, so per-trial updates cost zero parameter passes, and (iii) keeps the action space open-ended.

A.5 Process Supervision

Process supervision densifies a sparse outcome signal by attaching intermediate scalar rewards to individual reasoning or decision steps. This idea spans modern language-model verification and test-time search (Uesato et al., 2022; Cobbe et al., 2021; Lightman et al., 2024; Wang et al., 2024b; Luo et al., 2024; Zhang et al., 2024; Guo et al., 2025; Snell et al., 2025; Zhang et al., 2025; Khalifa et al., 2026; Zheng et al., 2025), as well as classical reinforcement-learning densification (Ng et al., 1999; Andrychowicz et al., 2017; Arjona-Medina et al., 2019; Vezhnevets et al., 2017; Pathak et al., 2017; Burda et al., 2019; Christiano et al., 2017). However, all these methods assume scalar feedback consumed via gradient-based parameter updates. We extend process supervision to the verbal, in-context regime: the densified signal is typed natural-language critique consumed without any parameter update.

A.6 Multi-Agent Debate

A line of work casts inference-time reasoning as deliberation between multiple LLM instances, either as peers exchanging arguments to converge on a more reliable answer (Du et al., 2024; Liang et al., 2024) or as evaluators substituting for human judges (Zheng et al., 2023; Bai et al., 2022; Lee et al., 2024), with broader role-decomposition extending to multi-agent software- and task-execution frameworks (Wu et al., 2024; Qian et al., 2024; Li et al., 2023; Ma et al., 2025). These methods treat experts as interchangeable reasoners differentiated only by prompt persona, and the quantity they produce is a single converged judgment over a shared query. The PRISM panel inside MDForge departs on both counts: each expert is anchored to a fixed physics subsystem (force field, sampling, analysis) with non-overlapping jurisdiction, and the output is a typed, subsystem-attributable critique that names which part of the pipeline to edit rather than a consensus verdict, with a slow cross-task loop reweighting per-expert reputations from their pre-trial vs. post-execution consistency.

B Method Complement

This appendix complements the high-level description of MDForge in §4 with the three technical pieces a reader needs to reproduce the system: the full PRISM pseudocode, a per-agent summary of the prompts that drive every LLM call together with the debate protocol and aggregator, and the reputation update. Verbatim prompts are released with the code.

B.1 PRISM Pseudocode

See Algorithm 1.

B.2 Multi-Agent Debate Protocol

We present the debate protocol and aggregator that turn per-expert votes into a single typed critique. The prompt of each expert agent is deferred in Appendix F. PRISM uses $J{=}3$ specialists with fixed jurisdictions: *Force Field*, *Sampling*, and *Analysis*. The Analysis expert also covers restraints, standard-state correction, and thermodynamic-cycle algebra.

Shared expert output contract. Every expert returns the same JSON schema across design recommendation, pre-execution review, and benchmark post-execution review: a label $\ell_i \in \{\textsc{Pass}, \textsc{Uncertain}, \textsc{Fail}\}$, confidence $\kappa_i \in [0, 1]$, a load-bearing *strategic_insight* field, a list

Algorithm 1 MDForge's verbal RL update.

Require: task T (host + guest set); expert panel $E = \{e_{\mathrm{FF}}, e_{\mathrm{Samp}}, e_{\mathrm{Anal}}\}$; successful-trial budget N; failed-revision cap M; K=4 stages

$d_{\text{rec}} \leftarrow$ DESIGNDISCUSSION(E, T) ▷ Phase 0: one-off, web-search enabled
$\pi \leftarrow$ CODEAGENT.PROPOSE(T, d_{rec})
$n \leftarrow 0$; $m \leftarrow 0$; $H \leftarrow \emptyset$; $\pi^* \leftarrow \perp$
while $n < N$ and $m < M$ and not converged **do**
  ▷ *Phase 1: static and mechanical screening*
  $c_{L1} \leftarrow$ LAYER1.VERIFY(π)
  if c_{L1}.label = FAIL **then**
    $m \leftarrow m + 1$; $\pi \leftarrow$ CODEAGENT.REVISE$(\pi, [c_{L1}])$; **continue** ▷ bypass panel
  end if
  $\pi \leftarrow$ ENGINEER.DEBUG(π, T) ▷ mechanical fixes only; no methodological changes
  ▷ *Phase 2: pre-execution verbal screening*
  $(c_{\text{pre}}, S_{\text{pre}}) \leftarrow$ AGGREGATE$(\{e_i.\text{PREEVAL}(\pi)\}_i)$ ▷ advisory
  ▷ *Phase 3: engineer-led execution*
  $D \leftarrow$ ENGINEER.RUN(π, T) ▷ debug + run stages $1..K$ in sandbox
  if any required stage fails or no finite $\hat{\Delta G}$ is produced **then**
    $m \leftarrow m + 1$; $\pi \leftarrow$ CODEAGENT.REVISE$(\pi, [c_{L1}, c_{\text{pre}}, D])$; **continue**
  end if
  $B \leftarrow$ MULTIMOLECULEBENCHMARK(π, T) ▷ parallel over guest set
  $(c_{\text{post}}, S_{\text{post}}) \leftarrow$ AGGREGATE$(\{e_i.\text{POSTEVAL}(\pi, B)\}_i)$
  ▷ *Phase 4: sparse outcome reward*
  $r^* \leftarrow -\,\mathrm{MAE}(B)$; $n \leftarrow n + 1$; $m \leftarrow 0$
  if c_{post}.label = PASS and $S_{\text{post}} \geq \theta_{\text{pass}}$ **then**
    $\pi^* \leftarrow \pi$; **break**
  end if
  ▷ *Phase 5: in-context Code agent update*
  $K_t \leftarrow \{c_{L1}, c_{\text{pre}}, c_{\text{post}}\}$; $H \leftarrow H \cup \{D, B, r^*\}$
  $\mathcal{C}_t \leftarrow \{T,\ \pi,\ K_t,\ H\}$ ▷ K_t = critique set; H = prior trial summaries
  $\pi \leftarrow$ CODEAGENT.REVISE$(\pi, \mathcal{C}_t)$
  ▷ *Phase 6: per-task slow-loop reputation*
  $\rho \leftarrow$ UPDATEREPUTATIONS$(\rho, c_{\text{pre}}.\text{votes}, c_{\text{post}}.\text{votes})$
end while
return π^* if converged, else last runnable π

of severity-scored concerns, and a short reasoning synthesis. Cross-domain commentary is allowed only when another subsystem directly affects the expert's jurisdiction.

Debate protocol. Given a fixed panel of J experts, a single call to the panel runs two rounds. *(i)* Round 1 (independent, parallel). Each expert receives the task description and either the pipeline source (pre-execution) or the pipeline plus its execution results (post-execution), and emits an independent vote in the shared schema. All J experts run concurrently. *(ii)* Round 2 (cross-visibility, parallel). Each expert is shown the round-1 votes of the other $J-1$ experts and may revise its own vote in the same schema. All J experts again run concurrently.

The same two-round protocol is used in three modes: design recommendation (Phase 0, before any pipeline exists), pre-execution review (PREEVAL, after the Code agent emits π and the Engineer has removed mechanical faults), and benchmark post-execution review (POSTEVAL, after the benchmark returns per-guest $\hat{\Delta G}$).

Aggregator $\mathcal{A}_\rho$. The aggregator collapses the round-2 votes into the single typed critique c used by the Code agent. Each vote carries a label $\ell_i \in \{\text{PASS}, \text{UNCERTAIN}, \text{FAIL}\}$, a confidence $\kappa_i \in [0, 1]$, and a free-text strategic insight. The aggregator scores each label as $s(\text{PASS})=1$, $s(\text{UNCERTAIN})=0.5$, $s(\text{FAIL})=0$, and computes the reputation- and confidence-weighted mean

$$S = \frac{\sum_i \rho_i \, \kappa_i \, s(\ell_i)}{\sum_i \rho_i \, \kappa_i}, \tag{6}$$

which is collapsed to a single panel label via two thresholds: $S \geq 0.7 \Rightarrow$ PASS, $S \leq 0.3 \Rightarrow$ FAIL, otherwise UNCERTAIN. The accompanying critique text is built by concatenating the per-expert strategic-insight and concern strings in reputation-weighted order; the full per-expert transcript is also retained for the Code agent's next-trial context.

B.3 Reputation Update and Convergence

Each expert i carries a Beta posterior over its reliability, defined as the probability that its pre-execution vote is consistent with the post-execution outcome. The prior is uniform, $\mathrm{Beta}(\alpha_i{=}1, \beta_i{=}1)$. After each completed trial, the orchestrator inspects each expert's $(c_{\text{pre},t}, c_{\text{post},t})$ vote pair and applies a single Beta update per expert:

$$\begin{aligned} \alpha_i \ &+= \mathbf{1}[\text{expert } i \text{ consistent}], \\ \beta_i \ &+= \mathbf{1}[\text{expert } i \text{ inconsistent}], \end{aligned} \tag{7}$$

where "consistent" is the agreement between the pre-execution label of expert i and the post-execution outcome of the trial (operationalized as the agreement between i's pre-execution vote and the aggregator's post-execution label). The deterministic weight fed into the aggregator $\mathcal{A}_\rho$ is the posterior mean

$$\rho_i = \frac{\alpha_i}{\alpha_i + \beta_i}, \tag{8}$$

which is the maximum-likelihood estimate of reliability under the Beta–Bernoulli

model. A Thompson-sampling variant draws $\tilde{\rho}_i \sim \mathrm{Beta}(\alpha_i, \beta_i)$ per trial; the deterministic posterior-mean form is the default reported in the main results.

Convergence within a task. Under the Beta–Bernoulli update of Eq. (7), ρ_i converges almost surely to expert i's true reliability p_i as the number of consistent/inconsistent observations grows, with $\mathrm{Var}(\rho_i) = \rho_i(1-\rho_i)/(\alpha_i+\beta_i+1) = \mathcal{O}(1/n_i)$. Within the N=5-successful-trial budget of a single task (a trial is counted only when the agent produces a runnable pipeline), $n_i \leq 5$ per expert, so the posterior mean has not converged to p_i; the update therefore functions as a soft prior that prevents the aggregator from equal-weighting an obviously-miscalibrated expert with reliable ones. The empirical ρ_t trajectories per host are written to the per-task reputation log released with the code.

C Experimental Details

We provide the experimental details: LLM and MD configurations, hardware, trial protocol, and benchmark splits. Exact configurations and prompt texts are released with the code.

LLM configuration. We record, for each agent role in the MDForge loop (Code agent, Engineer agent, the J=3 panel experts, Layer-1 static verifier, and the aggregator $\mathcal{A}_\rho$), the model identifier, sampling temperature, maximum context, and retry policy. The same configuration is reused across all hosts; baselines are run on the same backbone with their own prompt templates. We use the Claude Opus 4.7 as the default backbone.

Hardware and compute budget. Each guest evaluation runs in $\approx$ 2 GPU-hours on a single A40 node. The verbal RL loop continues until N=5 successful trials accumulate per host (a trial is counted when the agent produces a runnable pipeline; attempts that abort at Layer-1, crash the Engineer, or are abandoned by the agent do not count toward N), so the total round count per host can exceed five.

Trial protocol. A trial begins with the Code agent emitting a pipeline and ends either when the pipeline aborts at a stage boundary or when it completes the analysis stage and returns a $\hat{\Delta G}$ per training guest. *Coding success* counts a trial whose emitted code is runnable, regardless of whether it later crashes or does not converge inside MD. The *best pipeline* reported in Table 1 is selected, among trials that complete the production stage on all four training guests, as the one with the highest training-set Kendall τ; the same selection rule is applied to all methods.

System	N total	Train indices	Held-out indices
SAMPL4 CB[7]	14	1, 5, 8, 10	0, 2, 3, 4, 6, 7, 9, 11, 12, 13
SAMPL4 OAH	9	0, 3, 5, 8	1, 2, 4, 6, 7
SAMPL5 CBClip	10	0, 2, 3, 4	1, 5, 6, 7, 8, 9

Table 5: **Benchmark train/test splits.** Indices are zero-based within each host's guest list.

Benchmark splits. For each of the three hosts (CB[7], OAH, CBClip), the guest set is split into four training guests (visible to the verbal RL feedback loop) and the remaining guests as a held-out test set. The training quadruple is chosen to span the experimental ΔG range of each host and is fixed across all methods to keep comparisons aligned. See Table 5 for details.

D Uncertainty on the Held-Out Rank Correlations

The held-out sets are small ($n_{\mathrm{held}} = 10$, 5, and 6 for CB[7], OAH, and CBClip), so a rank correlation can move on a few pairwise reorderings. Table 6 reports two robustness checks on the selected trial: a nonparametric bootstrap 95% confidence interval, and a leave-one-out (LOO) range obtained by dropping each held-out guest in turn and recomputing τ.

Host	Method	τ	LOO range	Bootstrap 95% CI
CB[7]	Step-level	0.16	$[-0.06, 0.61]$	$[-0.65, 1.00]$
	Trial-level	0.24	$[-0.06, 0.44]$	$[-0.60, 1.00]$
	MDForge	**0.56**	$[\mathbf{0.33}, \mathbf{0.78}]$	$[-0.18, 1.00]$
OAH	Step-level	0.20	$[-0.33, 0.33]$	$[-1.00, 1.00]$
	Trial-level	0.20	$[-0.33, 0.33]$	$[-1.00, 1.00]$
	MDForge	**0.20**	$[\mathbf{0.00}, \mathbf{0.67}]$	$[-1.00, 1.00]$
CBClip	Step-level	0.07	$[-0.40, 0.40]$	$[-1.00, 1.00]$
	Trial-level	0.20	$[-0.20, 0.40]$	$[-0.67, 1.00]$
	MDForge	**0.47**	$[\mathbf{0.20}, \mathbf{0.60}]$	$[-0.33, 1.00]$

Table 6: **Uncertainty on held-out Kendall τ.** Point estimate, leave-one-out range (drop one held-out guest, recompute), and nonparametric bootstrap 95% CI. At $n_{\mathrm{held}} = 5$–10 the bootstrap intervals are uninformative by construction; the LOO ranges carry the signal.

The bootstrap intervals span most of $[-1, 1]$ at this sample size and are essentially uninformative; we report them for completeness. The LOO range

is the sharper instrument here, because it perturbs the evaluation set exactly one guest at a time, which is the failure mode of concern. Under LOO the ordering is stable: on CB[7] the entire MDForge range (lower bound 0.33) lies above both baselines' point estimates (0.24, 0.16), and on CBClip the MDForge lower bound (0.20) meets or exceeds them (0.20, 0.07). The baselines are also less stable than MDForge, dipping below zero on CB[7] and OAH, i.e. dropping a single guest can flip them to anti-correlated, whereas MDForge stays non-negative on all three hosts. No single held-out guest is responsible for the reported gain.

E Reputation-Weighted vs. Equal-Weighted Aggregation

Table 7 isolates the reputation loop of §4.3 by replacing ρ_t with uniform expert weights, holding the panel, the stage diagnostics, and the code agent fixed. Reputation weighting gives a small but consistent gain (0.40 ± 0.14 vs. 0.33 ± 0.11 on CB[7]). We are explicit that within a five-trial budget it acts as a stabilisation component rather than the principal driver: the dominant contribution comes from the typed per-stage diagnostics and subsystem-scoped critique, whose removal costs far more (Table 1).

Aggregation	Trial					Mean ± std
	1	2	3	4	5	
Equal weighting	0.33	0.16	0.33	0.42	0.42	0.33 ± 0.11
Reputation	0.20	0.33	0.42	0.51	0.56	$\mathbf{0.40 \pm 0.14}$

Table 7: **Reputation-weighted vs. equal-weighted expert aggregation on CB[7].** Per-trial held-out Kendall τ over $N{=}5$ successful trials, with all other components held fixed.

F Agent Prompts

We present the system prompts that define each agent's role inside MDForge. Three domain experts (force-field, sampling, analysis) act as peer co-designers and reviewers, while a single code-writing agent, split between the Pipeline Writer and the Pipeline Engineer, is responsible for producing and debugging the four-stage Python pipeline. Each panel below distills the operative content of the corresponding system prompt; the full prompts are released with the code.

Force-Field Expert: system prompt summary

Persona. A senior computational chemist with 10+ years on small-molecule force-field development and on the parameterization of SAMPL3–9 host-guest benchmarks. The agent participates as a peer in pipeline design, not as a narrow gatekeeper.

Primary domain.

- *Guest force fields.* GAFF/GAFF2+AM1-BCC as SAMPL default, with known $\sim$1 kcal/mol over-binding bias on cation-π contacts; OpenFF (Sage/Parsley) for more robust SMIRNOFF typing on non-standard cations; CGenFF only when the host is also CHARMM-parameterized. Aware of the canonical antechamber failure where tertiary ammonium nitrogen is mistyped as `nz` (sp^2) instead of `n4`.
- *Host force fields.* CB7 is parameterized with the same family as the guest. Charge provenance of the SAMPL-distributed `cb7.mol2` is treated as suspect; regenerating with AM1-BCC for consistency is the most defensible option.
- *Water and ions.* TIP3P as the SAMPL baseline, with $\sim$0.5–1 kcal/mol bias toward less-negative ΔG around cationic ammoniums. OPC or TIP4P-Ew paired with matched ion sets (Joung-Cheatham for TIP3P/SPC; OPC-trained ions for OPC) for improved cation hydration. Never mix water-specific ions with the wrong water model.
- *File consistency.* Atom-type assignments must agree between mol2, frcmod and tleap; `parmchk2` must run after any atom-type rewriting; `tleap.log` should be scanned for "Could not find..." warnings.

Co-design behaviour. Before enumerating concerns, the agent commits to a strategic position: for the specific chemical class at hand, what force-field choice would it make and why. It then audits whether the Writer's choice is defensible by that standard. In post-eval, it reads anomalous energy components and the reported ΔG through the lens of class-specific systematic biases (e.g., GAFF2+AM1-BCC+TIP3P should land $\sim$1–2 kcal/mol less negative than the experimental reference for CB7-cation systems). Cross-domain remarks on sampling, restraints or analysis are welcome when they affect whether force-field concerns are even detectable.

Operating modes. The same agent is invoked in four modes: (A) design recommendation before any pipeline exists, (B) pre-eval critique of a proposed design, (C) post-eval interpretation of a single-molecule run, and (D) post-eval over a multi-molecule benchmark, where it proposes the specific, surgical pipeline edit (named file, region and change) that would most improve next-iteration MAE.

Output contract. Each invocation returns a single JSON object with fields `label` $\in$ {`pass`,`fail`,`uncertain`}, `confidence`, a load-bearing `strategic_insight`, a list of `concerns` (each with severity and suggested focus), and a final `reasoning` synthesis. The `strategic_insight` is asked to be method-level rather than parameter-level, system-aware, comparative across alternatives, and literature-grounded.

Sampling Expert: system prompt summary

Persona. A senior molecular-simulation methodologist with 10+ years of experience designing alchemical, APR and umbrella-sampling protocols for binding free energies. Calibrated on hundreds of SAMPL-style benchmarks and able to recognize an under-sampled protocol from the λ schedule alone.

Primary domain.

- *Strategy choice.* Two main families for absolute binding: Attach-Pull-Release (APR, Henriksen-Gilson; the SAMPL CB7 standard; typically 15–25 windows, 1–5 ns each) and alchemical absolute binding (less common on CB7, with known pitfalls around PME+decoupling in openmmtools, softcore LJ at α=0.5, electrostatics-first λ schedules, and dense LJ-endpoint spacing).
- *Integrator and timestep.* `LangevinMiddleIntegrator` as modern default; 2 fs with HBonds constraints, 4 fs with HBonds+HMR; 1/ps friction. 5 fs HMR is aggressive and requires validation.
- *Equilibration.* Minimize, then NVT heating (100–500 ps), then NPT density (0.5–2 ns; longer for charged guests). A pipeline that skips NPT is a red flag.
- *Replicates and seeds.* For CB7-class systems a single long replicate is often acceptable; each replicate must be seeded independently. ≥ 3 replicates is preferred but rarely fits the budget.
- *Hardware and wall-clock.* OpenMM CUDA or `pmemd.cuda`, never silently CPU. The agent is explicitly briefed on a 2-hour cap on the production stage and is told to surface the trade-off ("24 windows $\times$ 2 ns is defensible but exceeds budget; reduce to 12 or accept the violation") rather than demand the impossible.

Co-design behaviour. The `strategic_insight` field is required to answer three questions for the specific system class: (i) what sampling strategy is best practice (e.g., APR with Henriksen-Gilson corrections for CB7), (ii) is the Writer's choice defensible, and if it took the less-common alchemical route, why might that be reasonable, and (iii) what is the largest sampling-side risk to the reported ΔG under the chosen design and wall-clock budget. In post-eval, the agent reads wall-clock used vs. designed, integrator stability evidence, T/P/density drift, per-window dwell time, replicate scatter, autocorrelation, and MBAR overlap; cross-domain remarks on force-field or restraint choices are welcome when they affect sampling sufficiency.

Operating modes & output contract. As for the force-field expert, the same JSON schema is emitted in all four modes (design recommendation, pre-eval, single-molecule post-eval, multi-molecule benchmark post-eval). In Mode D the agent is asked to name a surgical pipeline edit (specific file, region, change) that should improve next-iteration MAE.

Analysis Expert: system prompt summary

Persona. A senior simulation methodologist whose specialty is statistical-mechanics estimators for free-energy calculations and the thermodynamic-cycle algebra around restraint application and release. Has implemented MBAR/BAR/TI from scratch, derived the Boresch standard-state correction from the Gaussian partition function, and spent years separating "the calculation didn't converge" from "the estimator was wrong" from "the restraint correction has the wrong sign".

Primary domain.

- *Restraint design.* APR-style (1 distance along the host symmetry axis; analytic release; standard for CB7) versus Boresch (6-DOF, closed-form correction, overkill for symmetric hosts). Anchors are rigid heavy atoms (host ring carbon or carbonyl centroid; guest bridgehead or ammonium N). Sane CB7 force constants: k_r=5–20 kcal/mol/Å^2, Boresch angles/torsions 50–200 kcal/mol/rad^2.
- *Standard-state correction.* The agent is given the harmonic well integral $V_{\text{well}}=(2\pi kT/k)^{3/2}$ and $\Delta G_{\text{release}}= -kT\ln(V_{\text{std}}/V_{\text{well}})$ with V_{std}=1660 Å^3, and is explicitly warned that sign errors, leg misplacement, or a missing $-RT\ln(n_{\text{sym}})$ symmetry factor are the most common cause of ΔG_{bind} off by 5–15 kcal/mol.
- *Estimator choice.* MBAR by default for absolute binding via decoupling; BAR/TI when only adjacent pairs or $\langle\partial H/\partial\lambda\rangle$ are available; the Henriksen-Gilson three-term decomposition for APR; MM-PB/GBSA only as a rough first pass.
- λ *schedule.* Electrostatics-first, soft-core LJ with α=0.5, denser LJ spacing near the endpoint, 8–12 electrostatics windows plus 12–15 LJ windows, MBAR overlap ≥ 0.03 between adjacent states.
- *Uncertainty.* Three layers: within-replicate (pymbar bootstrap/block-jackknife), across-replicate scatter, and systematic biases (force field, restraint correction, sampling); reporting ± 0.1 kcal/mol on a CB7 system is treated as a smell.

Co-design behaviour. The `strategic_insight` field is asked to answer: which estimator and λ schedule are right for the specific system, is the thermodynamic cycle algebra correctly implemented *in code* (term-by-term sign check), and what is the biggest analysis-side risk to the reported ΔG. The agent is given a numerical reasonableness band for CB7-adamantylammonium (literature $\Delta G \approx -14$ kcal/mol; GAFF2+AM1-BCC+TIP3P should land in $[-14, -10]$; values outside $[-18, -8]$ have something wrong).

Operating modes & output contract. As for the other experts: four-mode invocation and the shared JSON schema with `label`, `confidence`, `strategic_insight`, `concerns`, and `reasoning`.

Pipeline Writer: system prompt summary

Pipeline Writer (code-authoring agent).

Mandate. Author a complete, runnable MD pipeline that computes the binding free energy of the host-guest pair specified by the user, by writing Python code from scratch as a sequence of $K{=}4$ sequential stages. Docking is out of scope: a bound-complex `complex.pdb` is pre-staged in the working directory and stage 01 reads it directly.

Step 0: literature reconnaissance. Before writing a single line of code, the agent is required to use `WebSearch` and `WebFetch` for at least three queries combining the host class with terms such as "binding free energy method", "SAMPL benchmark", "attach-pull-release" and "alchemical absolute binding", and to fetch at least one methods paper. The `RATIONALE` paragraph must name the chosen method, cite at least one published reference for the choice on this host class, and explain why the method fits the system's physics. Methodological choices without a literature citation are not acceptable.

Four-stage pipeline. (1) `01_prep.py`: read `complex.pdb`, pick force field/charge model/water, parameterize the guest via `antechamber+parmchk2`, build the solvated topology in a single `tleap` call, then minimize. (2) `02_equilibrate.py`: short NVT heating + NPT density, reporting T/P/density traces and drift. (3) `03_production.py`: the expensive sampling stage, reading per-window sampling length from the `MDFORGE_PRODUCTION_NS_PER_WINDOW` environment variable. (4) `04_analysis.py`: MBAR/TI/WHAM/BAR with restraint standard-state correction and $-RT\ln(n_{\mathrm{sym}})$ symmetry correction, populating `delta_g_kcal_per_mol` with an uncertainty.

Molecule-agnostic invariant. A single pipeline must run on every (host, guest) task with only the input files changing. The agent is forbidden from hardcoding guest names, atomic charges, symmetry numbers, or pH, and must read those from `task_metadata.json`; only the canonical filenames `guest.mol2`, `host.mol2`, `complex.pdb` may appear in code.

Output contract. The reply must follow an exact `RATIONALE` / `ENTRY` / `FILE` block structure that the harness parses mechanically; deviations cause an automatic Layer-1 failure. Each stage must write `stage_NN_result.json` on exit (even on failure) with `status`, `wall_time_seconds`, `delta_g_kcal_per_mol`, `convergence_flags`, `energy_components`, `diagnostics`, and a `writer_notes` field that the verifier experts read.

Engineering pitfalls embedded in the prompt. The Writer is briefed on environment-specific failure modes: openmmtools' PME-with-decoupled-electrostatics incompatibility (must set `annihilate_electrostatics=True`), antechamber's `nz` vs. `n4` mis-typing on protonated tertiary amines (with the explicit warning that AM1-BCC charges on ammonium N are physically negative, so naive "$N{>}0$" sanity checks must not be inserted), `tleap` sourcing order (small-molecule FF before water leaprc), CB7 host-charge provenance, the requirement that result files be written before re-raising on failure, hard wall-clock budgeting for stage 03, mandatory GPU use, and a single-GPU-per-molecule invariant (no `ProcessPoolExecutor` across GPUs inside one pipeline).

Pipeline Engineer: system prompt summary

Pipeline Engineer (debug-and-run agent).

Mandate. Given the four stage files the Writer just emitted, debug, iterate, and run the pipeline end-to-end in the sandbox until stage 04 outputs a finite ΔG. The session ends with a non-null `delta_g_kcal_per_mol` or it is considered failed; the next trial inherits worse context if it fails.

Execution discipline. Stages are run sequentially with synchronous blocking `Bash` calls (`timeout` set generously). After each run the agent inspects `stage_NN_result.json` before proceeding. Stage 03 must not be re-run once it has succeeded: if stage 04 then crashes, only stage 04 is re-run against the existing per-window energies.

Hard rules (allowed vs. forbidden edits). The Engineer may make mechanical fixes (switching `antechamber -c bcc`→`-c rc` to skip a hang, removing a wrong sanity check, fixing `tleap` sourcing order, adding `os.makedirs(..., exist_ok=True)`, catching format exceptions). It is forbidden from (i) changing methodological choices (APR→DDM, GAFF2→OPLS, TIP3P→OPC, reducing production length), (ii) gaming QC gates to manufacture success (silencing flags, zeroing uncertainty, lowering thresholds), (iii) introducing molecule-specific hardcoding, or (iv) bypassing `MDFORGE_PRODUCTION_NS_PER_WINDOW` to make ΔG look better in the debug pass.

Forbidden tools. The Engineer runs in a single, non-resumable session. `ScheduleWakeup`, `Monitor`, `ToolSearch`, and `run_in_background: True` are all explicitly disabled; the only available tools are `Read`, `Write`, `Edit`, `Bash`. Long-running stages are handled by blocking `Bash` with a large timeout rather than backgrounding.

Exit conditions. Success: `stage_04_result.json` carries a finite numeric ΔG and all earlier stages completed cleanly. Time budget: $\sim$60 minutes wall-clock for the whole debug session; the agent wraps up gracefully if running short. Stuck: if the same kind of fix has been tried twice on the same stage and the same error keeps coming back, the agent stops and documents the blocker for the next trial's Writer to handle.